



۱. کد زیر مربوط به سیستم انتقال وجه بانک میوبانک است که به تازگی در اثر حمله سایبری مقدار زیادی پول دزدیده شده است. وظیفه شما که به عنوان برنامه نویس در این بانک کار می کنید، این است که مشکل نفوذ را حل کنید (نوشتن کد صحیح قسمت مشکل دار اجباریست) و به همراه مستندات فنی که چند جمله برای آن کفایت می کند، برای مدیر امنیتی میوبانک ارسال کنید. اگر به جز مشکل نفوذ گفته شده راهکار یا طرح مفیدی برای بهبود این سیستم ارائه دهید، به شما پاداش (نمره امتیازی) تعلق می گیرد و برعکس اگر ایده شما منجر به کم شدن امنیت سیستم یا ایجاد نفوذ جدید شود، ممکن است جریمه شوید. (نمره منفی)

```
1. class DigitalWallet {
2.     public:
3.         double balance;
4.
5.         DigitalWallet(double initial_balance) {
6.             balance = initial_balance;
7.         }
8.
9.         void transfer(DigitalWallet& target_wallet, double amount) {
10.            balance -= amount;
11.            target_wallet.balance += amount;
12.        }
13.
14.        double get_balance() {
15.            return balance;
16.        }
17.    };
```

مشکل اصلی سیستم انتقال وجه چک نکردن موجودی حساب هست و باعث می شود که امکان انتقال وجه بیشتر از موجودی حساب امکان پذیر باشد.

قرار دادن همین یک خط کد به تنهایی در تابع transfer برای قسمت اجباری سوال کافیست: `11. if (balance < amount) return;` به سایر راهکار های مرتبط با Encapsulation که باعث بهبود سیستم شود، نمره امتیازی تعلق می گیرد.

۲. شما در یک شرکت دانش بنیان مشغول به باز طراحی و مهندسی معکوس یک دستگاه قهوه ساز هوشمند خارجی هستید. بعد از ساخت نمونه اولیه در فرآیند تست طولانی مدت متوجه می شوید که دستگاه شرکت شما برخلاف مدل خارجی خیلی زود خراب می شود. پس از بررسی های فراوان مشخص می شود که مشکل خرابی زود هنگام به دلیل ایده برنامه نویس قبلی برای افزایش سرعت تولید قهوه دستگاه بوده که با غیر فعال کردن شستشوی خودکار دستگاه، باعث سوختن زود هنگام موتور های گران قیمت دستگاه شما می شده است. بعد از این اتفاق تیم نرم افزاری شرکت تصمیم می گیرد ساختار شی گرایی این دستگاه باز نویسی و دسترسی ها محدود شوند تا مشکلاتی از این قبیل در آینده مجدد اتفاق نیفتد.

کد زیر ساختار جدید کلاس دستگاه شماست که باید تکمیل شود:

```

1. class SmartCoffeeMaker {
2.     private:
3.         int water_ml;
4.         int coffee_beans_g;
5.         int cups_made;
6.     public:
7.         SmartCoffeeMaker(int water, int beans) : water_ml(water), coffee_beans_g(beans),
            cups_made(0) {}
8.
9.         void make_espresso();
10.        void clean_machine();
11. };

```

قوانین دستگاه:

- هر شات اسپرسو دقیقاً ۳۰ میلی لیتر آب و ۱۵ گرم دانه قهوه مصرف می‌کند.
- دستگاه باید بعد از ساخت ۳ فنجان اسپرسو قفل شود و تا تابع شستشو یعنی `clean_machine()` صدا زده نشود، هیچ کار دیگری نکند.
- در صورتی که دستگاه نیاز به شستشو داشته باشد و یا مواد اولیه آن برای ساخت قهوه کم باشد، باید پیغام مناسب را چاپ کند.

وظیفه شما:

۱. نمودار (UML) کلاس را رسم کنید. (رعایت استاندارد های سطح دسترسی الزامی است!)
۲. تابع `SmartCoffeeMaker::make_espresso()` را با توجه به قوانین دستگاه که بالا نوشته شد، پیاده سازی کنید به گونه ای که اگر تابع `main()` زیر اجرا شود، خروجی مورد انتظار را چاپ شود.

```

1. int main() {
2.     SmartCoffeeMaker machine(130, 70);
3.
4.     machine.make_espresso();
5.     machine.make_espresso();
6.     machine.make_espresso();
7.     machine.make_espresso();
8.     machine.clean_machine();
9.     machine.make_espresso();
10.    machine.make_espresso();
11.
12.    return 0;
13. }

```

خروجی مورد انتظار:

```

Espresso made
Espresso made
Espresso made
Error: Machine is not clean
Espresso made
Error: Not enough ingredients

```

۳. امتیازی: به صورت خلاصه توضیح دهید چگونه این طراحی و کپسوله سازی متغیر ها باعث رفع خطرات احتمالی می‌شود.

```

void SmartCoffeeMaker::make_espresso() {

    if (cups_made >= 3) {
        std::cout << "Error: Machine is not clean" << std::endl;
        return;
    }
    if (water_ml < 30 || coffee_beans_g < 15) {
        std::cout << "Error: Not enough ingredients" << std::endl;
        return;
    }
    water_ml -= 30;
    coffee_beans_g -= 15;
    cups_made++;
    std::cout << "Espresso made" << std::endl;
}

```

SmartCoffeeMaker

- water_ml: int - coffee_beans_g: int - cups_made: int
+ SmartCoffeeMaker(int water, int beans) + make_espresso(): void + clean_machine(): void

